

Visualization with Matplotlib

AI teams

School of Mathematics and Physics
Xi'an Jiaotong-Liverpool University

May 5, 2026

Why Use Matplotlib?

- Visualize data distributions and trends.
- Plot training and validation loss curves.
- Display images, matrices, and feature maps.
- Inspect model inputs and outputs.

Basic Usage of `plt.plot`

`plt.plot()` is used to draw line plots and point plots.

```
plt.plot(x, y)
```

Here, x controls the horizontal axis and y controls the vertical axis.

Basic line plot

```
import matplotlib.pyplot as plt
```

```
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]
```

```
plt.plot(x, y)
plt.show()
```

This connects the points in order:

 $(1, 2), (2, 4), (3, 6), (4, 8), (5, 10)$

as x.

1

Only y values

```
loss = [1.5, 1.1, 0.8, 0.6, 0.5]
```

```
plt.plot(loss)
plt.show()
```

[illegible]

Implicit x values

```
x = [0, 1, 2, 3, 4]
y = [1.5, 1.1, 0.8, 0.6, 0.5]
```

```
plt.plot(x, y)
plt.show()
```

Common Options in `plt.plot`

`plt.plot()` has many useful options.

Option	Meaning
<code>color</code>	line or marker color
<code>linestyle</code>	style of the line
<code>marker</code>	style of the data point marker
<code>linewidth</code>	thickness of the line
<code>markersize</code>	size of the marker
<code>label</code>	name used in the legend

Marker Style

Markers show the position of individual data points.

Circle markers

```
plt.plot(x, y, marker="o")  
plt.show()
```

Common marker options:

Marker	Meaning
"o"	circle
"s"	square
"^"	triangle up
"x"	x marker
"*"	star
"."	point

Line Style

Line style controls how data points are connected.

Dashed line

```
plt.plot(x, y, linestyle="--")  
plt.show()
```

Common line styles:

Line style	Meaning
"_"	solid line
"-"	dashed line
":"	dotted line
"-."	dash-dot line
"None"	no connecting line

Color

The `color` option controls the color of the line and marker.

Set line color

```
plt.plot(x, y, color="red")  
plt.show()
```

Common color choices:

Code or name	Color
"r" or "red"	red
"g" or "green"	green
"b" or "blue"	blue
"k" or "black"	black
"orange"	orange

Line Width and Marker Size

`linewidth` controls line thickness.

`markersize` controls marker size.

Control line and marker size

```
plt.plot(x,y,  
         marker="o",  
         markersize=8)
```

```
plt.plot(x,y,  
         linestyle="-",  
         linewidth=2)
```

Use larger values when the figure will be used in slides or reports.

Using Label and Legend

The `label` option gives a name to each curve.

Plot with label

```
plt.plot(x, y, label="Training loss")
```

```
plt.legend()
```

```
plt.show()
```

`plt.legend()` displays the labels in the figure.

Without `plt.legend()`, the labels are defined but not shown.

Plotting Multiple Curves

Multiple curves can be plotted in the same figure.

Training and validation loss

```
epochs = [1, 2, 3, 4, 5]

train_loss = [1.2, 0.9, 0.7, 0.55, 0.43]
val_loss = [1.3, 1.0, 0.82, 0.70, 0.68]

plt.plot(epochs, train_loss, marker="o", label="Training loss")
plt.plot(epochs, val_loss, marker="s", label="Validation loss")

plt.legend()
plt.show()
```

Different markers help distinguish curves.

Combining Style Options

A professional plot usually combines several options.

Styled plot

```
plt.plot(epochs,train_loss,color="blue",
         linestyle="-",marker="o",linewidth=2,
         markersize=6,label="Training loss"
)

plt.plot(
    epochs,val_loss,color="red",
    linestyle="--",marker="s",linewidth=2,
    markersize=6,label="Validation loss"
)

plt.legend()
plt.show()
```

Why Add Axis Labels?

Axis labels explain what the horizontal and vertical axes represent.

- The x-axis label tells what variable is being changed.
- The y-axis label tells what quantity is being measured.
- Without labels, the figure can be ambiguous.
- Labels make the figure understandable without extra explanation.

For a loss curve:

x-axis: epoch
y-axis: loss

Adding Axis Labels

Use `plt.xlabel()` and `plt.ylabel()`.

Axis labels

```
plt.plot(epochs, train_loss, label="Training loss")
```

```
plt.xlabel("Epoch")
```

```
plt.ylabel("Loss")
```

```
plt.legend()
```

```
plt.show()
```

Good labels should be short, clear, and include units when relevant.

Why Add a Legend?

A legend explains what each curve represents.

- Necessary when the figure contains multiple curves.
- Helps compare training and validation behavior.
- Makes the plot readable without looking at the code.
- Avoids confusion when curves have similar shapes.

Each curve needs a `label`, and the figure needs `plt.legend()`.

Adding a Title

A title summarizes the purpose of the figure.

Title

```
plt.plot(epochs, train_loss, label="Training loss")
plt.plot(epochs, val_loss, label="Validation loss")

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training and Validation Loss")
plt.legend()

plt.show()
```

Use informative titles, not overly long titles.

Adding Grid Lines

Grid lines help read values from the figure.

Grid

```
plt.plot(epochs, train_loss, label="Training loss")
plt.plot(epochs, val_loss, label="Validation loss")

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)

plt.show()
```

Grid lines are especially useful for training curves and numerical comparisons.

Controlling Axis Range

Use `plt.xlim()` and `plt.ylim()` to control the displayed range.

Axis range

```
plt.plot(epochs, train_loss, label="Training loss")
plt.plot(epochs, val_loss, label="Validation loss")

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.xlim(1, 5)
plt.ylim(0, 1.5)
plt.legend()

plt.show()
```

This is useful when comparing figures using the same scale.

A Complete Good Figure Example

Recommended structure

```
import matplotlib.pyplot as plt
epochs = [1, 2, 3, 4, 5]
train_loss = [1.2, 0.9, 0.7, 0.55, 0.43]
val_loss = [1.3, 1.0, 0.82, 0.70, 0.68]

plt.figure(figsize=(6, 4))
plt.plot(epochs, train_loss,
         marker="o", linestyle="-",
         linewidth=2, label="Training loss")
plt.plot(epochs, val_loss,
         marker="s", linestyle="--",
         linewidth=2, label="Validation loss")

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training and Validation Loss")
plt.legend()
plt.grid(True)
plt.show()
```

What Makes a Good Plot?

A good plot should be **clear, readable, and self-contained**.

- Use **meaningful axis labels**.
- Use a **legend** when plotting multiple curves.
- Use **different markers or line styles** for different curves.
- Use sufficient line width for readability.
- Use a grid when numerical comparison is important.
- Avoid too many curves in one figure.
- Keep the title concise and informative.

Saving a Figure

Use `plt.savefig()` to save the current figure to a file.

Save figure

```
plt.plot(epochs, train_loss, label="Training loss")
plt.plot(epochs, val_loss, label="Validation loss")

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training and Validation Loss")
plt.legend()
plt.grid(True)

plt.savefig("loss_curve.png", bbox_inches="tight")
plt.show()
```

- "loss_curve.png" denotes that you save the figure to the same directory as your code file and you name it as 'loss_curve.png'.
- `bbox_inches="tight"`: removes unnecessary white margins.
- Common file formats: .png, .jpg, .pdf.